

Model Serving & Inference Optimization: Triển khai, Đo lường & Tối ưu Stack LLM Thực Chiến

Tập trung
Ghi chú
Hỗ trợ
Tiếng Việt

2

Quay lạiTiếp

1. Thuật ngữ cần biết

[Bản đồ Lab](#)

Đọc trước khi bắt đầu

240 phútTrung cấp

Học viên trực tiếp biến chiếc laptop của mình thành một AI serving node thực thụ, thực hiện đo đạc benchmark chi tiết (TTFT, TPOT, P50/P95/P99), kiểm chứng cơ chế Continuous Batching dưới áp lực tải 50 concurrent users, và phân tích cơ chế tối ưu sâu ở tầng kiến trúc vi xử lý.

Bài này đang nói về điều gì?

Phân biệt TTFT (prefill) và TPOT (decode) trong vòng đời sinh token của LLM

Bản chất toán học và trade-off của Quantization (GGUF k-quants vs Unsloth Dynamic)

Cơ chế Continuous (In-flight) Batching so với Static Batching truyền thống

Ứng dụng Định luật Little (Little's Law) để nhận diện điểm bão hòa hàng đợi (Queue Saturation)

Khái niệm Goodput@SLO và chiến lược phân bổ Thread trên kiến trúc CPU không đối xứng (Performance/Efficiency cores)

Hiện tượng bùng nổ độ trễ prefill $O(N^2)$ trong RAG context dài và sự cần thiết của Disaggregated Prefill/Decode

- Từ định danh năng lực phần cứng (Hardware Probe) đến lựa chọn runtime và mô hình nén phù hợp
- Từ đo đạc vi mô (Micro-benchmarking TTFT/TPOT/Threads) đến kiểm thử vĩ mô (Macro-serving & Load testing)
- Từ phân tích dữ liệu hiệu năng thực tế đến xây dựng luận điểm tối ưu có cơ chế kỹ thuật rõ ràng

Buổi Lab diễn ra như thế nào?

20 phút Học viên

Phase 0: Thiết lập Hạ tầng & Tải Runtime/Model

Quét thông số phần cứng, tải prebuilt binary llama.cpp tích hợp backend gia tốc và tải các phiên bản model GGUF.

120 phút Học viên

Phase 1: Base Track — Đo lường, Phục vụ & Tích hợp RAG

Chạy baseline benchmark, sweep thread count, dựng server OpenAI-compatible, thực hiện load test 10/50 users, bắt metrics batching và chạy pipeline RAG.

60 phút Học viên

Phase 2: Bonus Track — Đo lường Bùng nổ Prefill $O(N^2)$

Khảo sát tốc độ prefill khi context length tăng từ 256 lên 8192 tokens để xác định ngân sách context tối ưu cho RAG.

40 phút Học viên

Phase 3: Tổng hợp Báo cáo REFLECTION & Xác thực nộp bài

Viết giải thích cơ chế vào REFLECTION.md, chụp đủ 5 ảnh minh chứng và chạy make verify đảm bảo pass 100%.

Kết thúc bài, bạn có gì?

Bộ bằng chứng hiệu năng thực tế: hardware.json, models/active.json, các file báo cáo benchmarks/*.md

Báo cáo chuyên sâu submission/REFLECTION.md phân tích cơ chế phần cứng và đánh đổi kiến trúc

5 ảnh chụp màn hình chứng minh luồng hoạt động thực tế của hệ thống

Chưa cần lo

Không cần GPU rời đắt tiền hay cụm máy chủ đám mây. Mọi đánh giá được chấm dựa trên độ chặt chẽ trong lập luận trước và sau khi tối ưu (Before vs After) trên chính phần cứng của bạn.

Thuật ngữ gốc Bản chất khái niệm Minh họa trực quan

TTFT (Time To First Token)	Thời gian từ lúc gửi prompt đến khi token đầu bạn gọi món ở nhà hàng tiên xuất hiện. Bị chi phối bởi thời gian nạp prompt (Prefill) và thời gian xếp hàng (Queue Wait).	Giống thời gian từ khi đến khi đĩa khai vị đầu tiên được dọn ra bàn.
-----------------------------------	---	--

TPOT (Time Per Output Token)	Thời gian trung bình để sinh ra mỗi token tiếp theo (Decode loop). Bị giới hạn bởi Băng thông bộ nhớ (Memory Bandwidth) vì phải đọc lại toàn bộ model weights cho mỗi token.	Tốc độ người bồi bàn liên tục rót từng ly nước tiếp theo sau khi đã bắt đầu phục vụ.
-------------------------------------	--	--

Continuous Batching	Kỹ thuật chèn thêm request mới vào slot trống ngay sau mỗi bước decode, thay vì bắt tất cả tải ở tầng 1 mới chạy một phải đợi request dài nhất hoàn thành như Static Batching.	Giống thang máy liên tục đón/trả người ở từng tầng thay vì phải chờ đủ mạch lên tầng thượng.
Goodput@SLO	Số lượng request mỗi giây (RPS) hoàn thành mà vẫn thỏa mãn cam kết chất lượng dịch vụ (SLO về TTFT/TPOT). Khác với <i>Raw Throughput</i> (chỉ đếm tổng số token bất chấp latency vọt lên hàng chục giây).	Bạn vận chuyển được 1000 gói hàng/ngày (Throughput) nhưng chỉ có 200 gói đến đúng hẹn trong 24h (Goodput@SLO).
Little's Law	Định luật toán học hàng đợi: $\$L = \lambda \times W\$$ (Số request trung bình trong hệ thống = độ đến $\lambda \times$ Thời gian xử lý trung bình). Dùng để tính Effective Concurrency .	Nếu quán ăn có 2 khách vào mỗi phút ($\lambda=2$) và mỗi người ngồi 30 phút ($W=30$), thì quán luôn có trung bình $\$L = 60\$$ khách đang ngồi/chờ.
Quantization (GGUF)	Kỹ thuật nén trọng số từ FP16/FP32 xuống các định dạng bit thấp hơn (4-bit, 2-bit) nhằm giảm dung lượng bộ nhớ và tăng tốc độ đọc từ RAM vào compute cores.	Giống như nén file ảnh chất lượng cao thành JPEG: dung lượng giảm 4 lần, tốc độ tải nhanh hơn nhiều nhưng có thể mất một chút chi tiết nhỏ.

Evaluated via Goodput@SLO
RequestLifecycle

+TTFT: Queue Time + Prefill Time(C

ompute/Memory bound)

+TPOT: Inter-Token Latency(Memory Bandwidth bound)

+E2ELatency: TTFT + TPOT *(N - 1)

ServerArchitecture

+ContinuousBatching: Dynamic slot packing

+PrometheusMetrics: tokens_predicted, n_busy_slots

+LittleLaw: Effective Concurrency = RPS * AvgLatency

VLearn Codelabs | Học AI thực chiến qua Lab

Model Serving & Inference Optimization: Triển khai, Đo lường & Tối ưu Stack LLM Thực Chiến

Tập trung

Ghi chú

Hỗ trợ

Tiếng Việt

2

Quay lạiTiếp

2. Mục tiêu & đầu ra

Bạn hoàn thành khi:

- Cài đặt thành công Runtime `llama.cpp` (prebuilt binary) và tải đầy đủ trọng số mô hình GGUF.
- Dựng hoàn chỉnh một Local Inference Server chuẩn OpenAI REST API (`/v1/chat/completions`) có tích hợp giám sát Prometheus `/metrics`.
- Đo đặc độc lập TTFT và TPOT dưới 2 cấp độ nén (`UD-Q4_K_XL` vs `UD-Q2_K_XL`) và các cấu hình phân luồng (`-t`).
- Kiểm chứng thực nghiệm cơ chế **Continuous Batching** và xác định chính xác điểm **Bão hòa hệ thống (System Saturation)** dưới tải Locust 10 & 50 users.
- Tích hợp thành công Serving Endpoint vào chu trình **RAG Pipeline (Embed $\rightarrow$ Retrieve $\rightarrow$ LLM)**.
- Khảo sát hiện tượng bùng nổ độ trễ prefill $O(N^2)$ khi mở rộng context length lên 8192 tokens.

- Chạy lệnh `make verify` (hoặc `.\lab.ps1 verify`) trả về **All checks passed (exit 0)** và commit toàn bộ kết quả vào public GitHub repo.

VLearn Codelabs | Học AI thực chiến qua Lab

Model Serving & Inference Optimization: Triển khai, Đo lường & Tối ưu Stack LLM Thực Chiến

Tập trung

Ghi chú

Hỗ trợ

Tiếng Việt

2

Quay lạiTiếp

3. Chuẩn bị

1. Clone Repository bài Lab:

<https://github.com/VinUni-AI20k/Day20-Track2-ModelServing-Lab> Mở terminal trên máy của bạn và chạy lệnh clone repository:

```
git clone
https://github.com/VinUni-AI20k/Day20-Track2-ModelServing-Lab.git
cd Day20-Track2-ModelServing
```

Copy

2. Điều kiện tối thiểu:

- **Hệ điều hành:** macOS (Apple Silicon M-series), Linux hoặc Windows 11.
- **Bộ nhớ:** Tối thiểu 8 GB RAM (Khuyến nghị 16 GB Unified Memory). Nếu máy có 4 GB RAM, chọn model nhỏ `Qwen3.5 0.8B`.
- **Dung lượng trống:** ~10 GB SSD trống.
- **Môi trường phần mềm:** Python `>= 3.10`, `git`, `curl`. (Không cần Docker, không cần compiler C++).

 Lưu ý đặc biệt cho người dùng Windows

Windows không có lệnh `make` mặc định. Lab đã cung cấp sẵn script `.\lab.ps1` thay thế hoàn toàn tương đương:

```
# Chạy một lần duy nhất khi bắt đầu để mở quyền script:
powershell -ExecutionPolicy Bypass -File labs/00-setup/bootstrap.ps1
```

```
# Cú pháp chạy tương đương make:
.\lab.ps1 probe      # Tương đương make probe
.\lab.ps1 setup      # Tương đương make setup
.\lab.ps1 bench      # Tương đương make bench
.\lab.ps1 serve      # Tương đương make serve
.\lab.ps1 verify     # Tương đương make verify
```

Copy

[!TIP] Khắc phục lỗi hiển thị Tiếng Việt & Decode trên Windows PowerShell: Trước khi thực hành, hãy chạy 3 dòng sau trong PowerShell để console hỗ trợ 100% chuẩn mã hóa UTF-8 quốc tế:

```
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8
chcp 65001
$env:PYTHONUTF8 = "1"
```

Copy

Hướng dẫn Cài đặt Runtime `llama.cpp` & Trọng số Model

`make setup / .\lab.ps1 setup`

1. Tạo `.venv` & Cài thư viện Python
2. Tự động tải Prebuilt `llama.cpp` (b10488)
3. Tải 2 bản Quantization GGUF từ Hugging Face

Thư mục `runtime/`

Thư mục `models/` & `models/active.json`

A. Cách cài đặt Tự động (Khuyến nghị):

Trên macOS / Linux:

```
make setup
```

-
- Copy

Trên Windows PowerShell:

```
.\lab.ps1 setup
```

-
- Copy

B. Cơ chế hoạt động của bước Setup:

1. Cài đặt Runtime `llama.cpp`:

- Lab sử dụng bản **Prebuilt Release Binary (build `b10488`)**, không cần compile mã nguồn.
- Script `labs/00-setup/fetch-runtime.py` tự động nhận diện phần cứng và tải bản build tối ưu:
 - **macOS Apple Silicon:** Tải `bin-macos-arm64` (~11 MB, tích hợp sẵn Apple Metal).
 - **Windows + NVIDIA:** Tải `bin-win-cuda-*-x64` (~140–240 MB, kèm CUDA runtime DLLs).
 - **Linux:** Tải `bin-ubuntu-x64` hoặc `bin-ubuntu-vulkan-x64` (~16–32 MB).
- Binary được giải nén vào thư mục `runtime/b10488/llama-b10488/` (gồm `llama-server` và `llama-bench`).

2. Lựa chọn & Tải Mô hình (Model Weights):

- Mặc định tải **Gemma 4 E2B** (~5.2 GB tổng cộng, cần ≥ 8 GB RAM):
 - Model chính (Primary 4-bit):
`gemma-4-E2B-it-UD-Q4_K_XL.gguf` (2.97 GB)
 - Model so sánh (Compare 2-bit):
`gemma-4-E2B-it-UD-Q2_K_XL.gguf` (2.24 GB)

Nếu máy có RAM < 8 GB hoặc muốn chạy cực nhanh: Hãy chuyển sang **Qwen3.5 0.8B** (~0.9 GB tổng cộng, cần ≥ 4 GB RAM) bằng cách thiết lập biến môi trường **trước khi chạy** `make setup`:

```
export LAB_MODEL=qwen35-0.8b      # macOS / Linux
$env:LAB_MODEL = 'qwen35-0.8b'    # Windows PowerShell
make setup                        # (hoặc .\lab.ps1 setup)
```

-
- Copy

[!TIP] **Tải thủ công khi bị chặn mạng trường (Hugging Face Blocked):** Nếu mạng bị chặn không tải tự động được, bạn có thể tải thủ công file GGUF bằng trình duyệt/IDM rồi đặt vào thư mục `models/`:

- Link tải Gemma 4 E2B Q4: [gemma-4-E2B-it-UD-Q4_K_XL.gguf](#)
- Link tải Gemma 4 E2B Q2: [gemma-4-E2B-it-UD-Q2_K_XL.gguf](#)

Hướng dẫn chụp và lưu Screenshot từ Terminal

Trong suốt bài lab, bạn cần chụp lại màn hình cửa sổ Terminal khi chạy các lệnh quan trọng và lưu đúng tên file vào thư mục `submission/screenshots/`:

Hệ điều hành Phím tắt chụp vùng Cách lưu file vào
chọn submission/sc

macOS	<code>Cmd + Shift + 4</code> (Kéo chọn vùng Terminal)	Ảnh mặc định ra Desktop <code>\$\rightarrow\$</code> Kéo thả vào <code>submission/screenshots/</code> hoặc dùng lệnh <code>mv ~/Desktop/Screenshot*.png submission/screenshots/<tên-file>.png</code>
Windows	<code>Win + Shift + S</code> (Kéo chọn vùng Terminal)	Dán vào Paint/Snip <code>\$\rightarrow\$</code> Save As định dạng PNG vào thư mục <code>submission\screenshots\<tên-file>.png</code>
Linux	<code>Shift + PrintScreen</code>	Lưu trực tiếp vào thư mục <code>submission/screenshots/<tên-file>.png</code>

[!CAUTION] Tên file ảnh phải tuyệt đối chính xác theo danh sách sau:

1. `01-hardware-probe.png` (Sau khi chạy `make probe` hoặc `.\lab.ps1 probe`)

2. 02-bench.png (Bảng kết quả sau khi chạy `make bench` hoặc `.\lab.ps1 bench`)
3. 03-serve-and-smoke.png (Cả 2 terminal `make serve` và `make smoke`)
4. 04-locust-10.png (Bảng kết quả Locust sau khi chạy `make load-10`)
5. 05-locust-50.png (Bảng kết quả Locust sau khi chạy `make load-50`)

Hướng dẫn cách Đọc Terminal để Điền Báo cáo (Reflection)

[!IMPORTANT] **Nguyên tắc phân tích: Terminal chính là "bảng điều khiển" chân thực nhất của bạn.** Không đoán mò số liệu! Mỗi khi chạy xong một target, hãy quan sát trực tiếp output trên Terminal, đọc các số đo cụ thể và đối chiếu theo bảng hướng dẫn sau để điền vào [submission/REFLECTION.md](#):

Target vừa chạy	Vị trí nhìn trên Terminal	Dữ liệu cần lấy	Điền vào mục nào trong REFLECTION.md
<code>make probe</code>	Khối Platform, CPU, RAM, GPU	Tên OS, Chip, Số Cores, Dung lượng RAM, Backend GPU	Mục §1: Hardware & runtime
<code>make bench</code>	Bảng so sánh 2 quant (<code>UD-Q4_K_XL</code> & <code>UD-Q2_K_XL</code>)	Load time, TTFT P50/P95, TPOT P50/P95, Decode tok/s	Mục §2: Đo lường (Copy nguyên bản)
<code>make tune</code>	Các dòng <code>-t <N> tg128 = <X> tok/s</code>	Tìm <code>-t</code> có tok/s cao nhất (Best) và so với <code>-t</code> mặc định	Mục §5: Single change that mattered most

<code>make load-report</code>	Bảng so sánh <code>Users, RPS, P50, P95, Eff. concurrency</code>	Độ tăng RPS, độ tăng P95, giá trị Concurrency so với slots	Mục §3: Serving under load
<code>make metrics</code>	Dòng <code>Highest sampled n_busy_slots_per_decode</code>	Giá trị peak busy slots (vd: 3.92/4)	Mục §3: Serving under load
<code>make pipeline</code>	Dòng <code>Mean over 3 queries (ms)</code>	Thời gian embed, retrieve, llm và % dominant stage	Mục §4: Integration
<code>make sweep-ctx</code>	Bảng quét context <code>256 -> 8192 tok</code>	Tốc độ prefill (tok/s) và TTFT ở 256 tok vs 8192 tok	Mục §6: Bonus track

VLearn Codelabs | Học AI thực chiến qua Lab

Model Serving & Inference Optimization: Triển khai, Đo lường & Tối ưu Stack LLM Thực Chiến

Tập trung

Ghi chú

Hỗ trợ

Tiếng Việt

2

Quay lạiTiếp

4. Thực hành

[!IMPORTANT] **Lưu ý quan trọng cho học viên:** Tất cả các bảng số liệu, kết quả terminal và biểu đồ minh họa bên dưới là **kết quả mẫu thực nghiệm đo trên một máy MacBook**. Khi thực hành, bạn **phải tự chạy các lệnh trên chính máy tính của mình** (dùng `make <target>` trên Mac/Linux hoặc `.\lab.ps1 <target>` trên Windows), quan sát output terminal thực tế để đưa ra phân tích của riêng bạn!

♦ Bước 0.1: Quét thông số phần cứng (`make probe / .\lab.ps1 probe`)

```
make probe          # macOS / Linux
.\lab.ps1 probe     # Windows PowerShell
```

Copy

Quan sát Terminal để lấy dữ liệu:

Nhìn vào khối thông tin in ra trên Terminal để xác định hệ điều hành, số nhân CPU, RAM và GPU backend:

```
Platform : Darwin 25.6.0 (arm64)
CPU       : Apple M5 (10 physical · 10 logical cores, extensions:
NEON)
RAM       : 16.0 GB
GPU       : apple_metal (Metal built into the release binary)
Model     : Gemma 4 E2B [LAB_MODEL=gemma4-e2b]
```

Copy

[!TIP] Hành động:

1. Chụp màn hình terminal lưu vào:
`submission/screenshots/01-hardware-probe.png`.
2. Mở file [submission/REFLECTION.md](#) $\rightarrow$ Điền thông tin vào **Mục §1 (Hardware & runtime)**.



(Hình 1: Ảnh chụp màn hình mẫu khi chạy `make probe`)

♦ Bước 0.2: Cài đặt Runtime & Model (`make setup / .\lab.ps1 setup`)

```
make setup          # macOS / Linux
.\lab.ps1 setup     # Windows PowerShell
```

Copy

Script tự động tạo `.venv`, tải prebuilt binary `llama-b10488` và tải 2 phiên bản GGUF từ Hugging Face:

- `gemma-4-E2B-it-UD-Q4_K_XL.gguf` (~2.97 GB — Model chính 4-bit)
- `gemma-4-E2B-it-UD-Q2_K_XL.gguf` (~2.24 GB — Model so sánh 2-bit)

♦ **Bước 1.1: Đo Baseline TTFT / TPOT & Quantization** (`make bench / .\lab.ps1 bench`)

```
make bench          # macOS / Linux
.\lab.ps1 bench     # Windows PowerShell
```

Copy

Quan sát Terminal để phân tích:

Terminal sẽ in ra bảng tổng kết latency baseline. Hãy copy bảng này và đọc 2 chỉ số cốt lõi:

- TTFT (Prefill):** Prompt xử lý mất bao lâu trước khi ra token 1?
- TPOT & Decode (tok/s):** Tốc độ sinh từng token tiếp theo nhanh hơn bao nhiêu lần?

Bảng kết quả mẫu tham khảo (Ví dụ trên MacBook):

Quantization	Size (GB)	Load (ms)	TTFT	TPOTE	P50/P95/P99 (ms)	Decode (tok/s)
			P50/P95	P50/P95		
			(ms)	(ms)		
UD-Q4_K_XL	2.97	4155	245 / 805	43.0 / 51.0	2953 / 3629 / 3629	23.3
UD-Q2_K_XL	2.24	3048	214 / 920	34.9 / 39.9	2411 / 2997 / 2997	28.7

[!NOTE] **Nhận xét chuyên sâu mẫu:** Bản UD-Q2_K_XL decode nhanh hơn **1.23× (tăng 23%)** và tiết kiệm **0.73 GB RAM**. Tuy nhiên trên máy 16 GB RAM, bản UD-Q4_K_XL vẫn là sweet spot cho production vì bảo toàn năng lực lập luận và độ mạch lạc của mô hình nhỏ (<7B). *(Hãy so sánh và nhận xét dựa trên con số thực tế máy bạn).*



(Hình 2: Ảnh chụp màn hình mẫu bảng kết quả `make bench`)

♦ **Bước 1.2: Tinh chỉnh Thread Scheduling** (`make tune / .\lab.ps1 tune`)

```
make tune          # macOS / Linux
.\lab.ps1 tune     # Windows PowerShell
```

Copy

Quan sát Terminal để tìm điểm tối ưu (Knee of the curve):

Quan sát các giá trị `tg128 (tok/s)` in ra cho từng mức luồng. Tìm xem mức `-t` nào cho thông lượng cao nhất và xem hiện tượng gì xảy ra khi tăng quá số luồng vật lý:

Bảng kết quả quét luồng mẫu tham khảo (Ví dụ trên MacBook):

Threads (`-t`) `tg128 (tok/s)` So với BestHiệu tượng kỹ thuật

1	25.9	96% Dưới ngưỡng bão hòa compute
5 (Best)	26.9	100% Điểm ngọt ngào (Sweet spot) tận dụng cụm P-cores
10 (Default)	23.7	88% Barrier synchronization penalty giữa P-core và E-core

20	22.0	82%	Oversubscription, tranh chấp L2 cache & context switching
----	------	-----	---

Throughput theo số lượng Thread (Ví dụ trên MacBook)-t 1-t 5 (Sweet spot)-t 10 (Default)-t 20 (Oversubscribed)3029282726252423222120Decode Throughput (tok/s)

[!IMPORTANT] **Cơ chế:** Nhiều CPU hiện đại (như Apple Silicon, Intel Gen 12+) có kiến trúc không đối xứng (**Nhân Performance + Nhân Efficiency**). Khi dùng tối đa số logical cores, các nhân P xung nhịp cao liên tục phải chờ nhân E chậm hơn đồng bộ sau mỗi phép nhân ma trận. Chọn đúng số luồng cho cụm P-core giúp tăng tốc đáng kể mà không cần compile lại.

♦ Bước 1.3: Dựng Server & Smoke Test (`make serve` + `make smoke`)

Terminal 1: Khởi động Server OpenAI-compatible:

```
make serve          # macOS / Linux
.\lab.ps1 serve     # Windows PowerShell
```

-
- Copy

Terminal 2: Gửi request kiểm thử tính năng và observability:

```
make smoke          # macOS / Linux
.\lab.ps1 smoke     # Windows PowerShell
```

-
- Copy

Quan sát Terminal:

Terminal 2 sẽ hiển thị câu trả lời completion và in ra bảng Prometheus metrics. Kiểm tra thấy `llamacpp:tokens_predicted_total > 0` là server đã hoạt động chuẩn xác!



(Hình 3: Ảnh chụp màn hình mẫu kết hợp giữa `make serve` và `make smoke`)

[!TIP] **Hành động:** Chụp ảnh màn hình (ghép hoặc để cạnh nhau 2 cửa sổ terminal) lưu vào: `submission/screenshots/03-serve-and-smoke.png`.

♦ Bước 1.4: Load Testing & Bắt Continuous Batching (`make load-10, make load-50, make metrics`)

Chạy tải 10 Users:

`make load-10` # (hoặc `.\lab.ps1 load-10`)

- 1.
2. Copy
3. 📸 Chụp ảnh terminal bằng Locust lưu vào:

`submission/screenshots/04-locust-10.png`.



(Hình 4: Ảnh

chụp màn hình mẫu bằng Locust tải 10 users)

4. Chạy tải 50 Users và Bắt Continuous Batching:

- Tại Terminal 2: `make load-50` (hoặc `.\lab.ps1 load-50`)
- Đồng thời tại Terminal 3 (chạy ngay khi Terminal 2 đang chạy): `make metrics` (hoặc `.\lab.ps1 metrics`)

Quan sát Terminal 3:

Terminal 3 sẽ scrape metrics liên tục và in ra dòng kết luận:

```
Highest sampled n_busy_slots_per_decode = 3.92 of 4 slots (98% Parallel Capacity)
```

Copy

[!TIP] **Hành động:**

1. Chụp ảnh màn hình terminal bằng Locust 50 lưu vào: `submission/screenshots/05-locust-50.png`.
2. Mở file [benchmarks/02-server-batching-u50.md](#) và điền nhận xét về hiệu quả gom batch của scheduler.

3. Điền giá trị `Peak n_busy_slots_per_decode` vào **Mục §3 trong REFLECTION.md**.



(Hình 5: Ảnh chụp màn hình mẫu bảng Locust tải 50 users)

♦ **Bước 1.5: Phân tích Bảo hòa Hệ thống** (`make load-report` / `.\lab.ps1 load-report`)

```
make load-report      # macOS / Linux
.\lab.ps1 load-report # Windows PowerShell
```

Copy

Quan sát Terminal để phân tích độ bão hòa (Định luật Little):

Đọc trực tiếp bảng so sánh giữa 10 users và 50 users:

1. **Offered load:** Tăng bao nhiêu lần (vd: 5×)?
2. **Throughput (RPS):** Thực tế tăng được bao nhiêu lần (vd: chỉ 1.37×)?
3. **P95 Latency:** Bị phình to bao nhiêu lần (vd: 2.55×)?
4. **Effective concurrency:** Đạt bao nhiêu so với 4 slots?

Bảng so sánh 10 Users vs 50 Users mẫu tham khảo (Ví dụ trên MacBook):

Chỉ số	10 Users	50 Users	Mức độ biến thiên
--------	----------	----------	-------------------

Offered Load	10 users	50 users	Tăng 5.00×
---------------------	----------	----------	-------------------

Delivered Throughput	0.70 RPS	0.95 RPS	Chỉ tăng 1.37× (Plateau)
-----------------------------	----------	----------	---

P50 Latency	13.0 s	30.0 s	Tăng 2.31×
--------------------	--------	--------	-------------------

P95 Latency	20.0 s	51.0 s	Bùng nổ 2.55×
--------------------	--------	--------	----------------------

Effective	8.6	28.2	Vượt 7.06× số slot
Concurrency (\$L\$)			phần cứng (4 slots)

Request Đến
(50 Users)
Hàng đợi Server
(Queue ~24 requests)
Continuous Batching
(4 Compute Slots / 3.92 Busy)
Phản hồi Client
(P95 = 51s -> Vi phạm SLO)

[!TIP] **Hành động:**

1. Mở file [benchmarks/02-server-results.md](#), xóa dòng required và điền luận điểm chứng minh server bị nghẽn ở **Queue Time**.
2. Copy các con số và giải thích vào **Mục §3 trong REFLECTION.md**.

♦ **Bước 1.6: Tích hợp RAG Pipeline** (`make pipeline / .\lab.ps1 pipeline`)

```
make pipeline      # macOS / Linux
.\lab.ps1 pipeline # Windows PowerShell
```

Copy

Quan sát Terminal để lấy Latency Breakdown:

Terminal sẽ in ra thời gian xử lý của từng stage qua 3 câu truy vấn:

- `embed`: 0.0 ms (stub)
- `retrieve`: 0.1 ms (stub)
- `llm`: 1436.1 ms (**Real**, chiếm **100.0% tổng độ trễ pipeline**)
- `total`: 1436.2 ms

[!TIP] **Hành động:**

- 1. Mở file [benchmarks/03-integration-results.md](#) $\rightarrow$ Khai báo rõ các module N16–N19 là Stub, N20 là Real.
- 2. Điền bảng phân tách latency và phương hướng tối ưu LLM vào **Mục §4 trong REFLECTION.md**.

♦ **Bước 2.1: Bonus B2 — Khảo sát Prefill Context Length** (make sweep-ctx / .\lab.ps1 sweep-ctx)

```
make sweep-ctx      # macOS / Linux
.\lab.ps1 sweep-ctx # Windows PowerShell
```

Copy

Quan sát Terminal để xem hiện tượng Attention $O(N^2)$:

Quan sát bảng prefill từ 256 tokens đến 8192 tokens. Chú ý mốc nào tốc độ prefill (tok/s) bắt đầu sụt giảm và TTFT tăng vọt:

Bảng chi phí Prefill theo Context Length mẫu tham khảo (Ví dụ trên MacBook):

Prompt Tokens	Prefill Throughput (tok/s)	TTFT Contribution (ms)	So với Tăng Tuyến Tính
256	882.2	290.2	1.00× (Linear)
1024	760.5	1346.4	1.16×
2048	804.2	2546.7	1.10×
4096	578.7	7077.7	1.52× (Quadratic Bend)
8192	477.1	17169.0	1.85× (Penalty nghiêm trọng)

Bùng nổ TTFT do Attention $O(N^2)$ theo Context Length (Ví dụ trên MacBook)
256 tok 1024 tok 2048 tok 4096 tok 8192 tok 181614121086420 TTFT Latency (Giây)

[!TIP] **Hành động:**

1. Mở file [benchmarks/bonus-ctx-len-sweep.md](#) và điền nhận xét về giới hạn ngân sách prompt trong RAG.
2. Điền số liệu before/after (256 tok vs 8192 tok) và phân tích cơ chế vào **Mục §6 trong REFLECTION.md (Bonus track)**.

VLearn Codelabs | Học AI thực chiến qua Lab

Model Serving & Inference Optimization: Triển khai, Đo lường & Tối ưu Stack LLM Thực Chiến

Tập trung
Ghi chú
Hỗ trợ
Tiếng Việt

2

Quay lại **Nội bài** để hoàn thành

Model Serving & Inference Optimization: Triển khai, Đo lường & Tối ưu Stack LLM Thực Chiến

Tập trung
Ghi chú
Hỗ trợ
Tiếng Việt

2

Quay lại **Tiếp**

5. Kiểm tra kết quả & Demo trên lớp

1. Tự kiểm tra tự động qua `make verify`:

Sau khi đã hoàn tất các bước và điền xong toàn bộ báo cáo, chạy lệnh xác thực:

Trên macOS / Linux:

`make verify`

-
- Copy

Trên Windows PowerShell:

```
.\lab.ps1 verify
```

-
- Copy

Quan sát Terminal để kiểm tra checklist:

```
==> Verifying submission readiness in
/Users/.../Day20-Track2-ModelServing-Lab
✓ Hardware probe: hardware.json present
✓ Model manifest: Gemma 4 E2B (UD-Q4_K_XL + UD-Q2_K_XL)
✓ Runtime: llama-server present (b10488)
✓ Latency baseline: benchmarks/01-quickstart-results.md valid
✓ Tuning sweep: benchmarks/01-tuning-tg128.md valid
✓ Load run at 10 users: benchmarks/locust-10_stats.csv valid
✓ Load run at 50 users: benchmarks/locust-50_stats.csv valid
✓ Saturation reading: benchmarks/02-server-results.md valid
✓ Continuous-batching evidence: benchmarks/02-server-batching-u50.md
valid
✓ RAG pipeline run: benchmarks/03-integration-results.md valid
✓ Reflection: submission/REFLECTION.md filled in
✓ Screenshots: 5 of 5 present
```

```
==> All checks passed! Ready to submit. (exit 0)
```

Copy

[!NOTE] Nếu `make verify` báo lỗi đỏ, Terminal sẽ in ra chính xác tên file còn thiếu hoặc placeholder chưa thay thế. Hãy đọc dòng thông báo lỗi đó để bổ sung!

Hướng dẫn Xử lý lỗi Decode / Encode trên Windows

Nếu chạy verify trên Windows gặp lỗi `UnicodeDecodeError: 'charmap' codec can't decode...` (do Windows đọc file có dấu tiếng Việt) hoặc `UnicodeEncodeError` (khi in icon ✓, ✗), hãy dùng 1 trong các cách sau:

Cách 1 (Nhanh nhất - Chạy Python UTF-8 trực tiếp):

```
python -X utf8 scripts\verify.py
```

- 1.
2. Copy

Cách 2 (Set biến môi trường UTF-8 trong PowerShell):

```
$env:PYTHONUTF8 = "1"
```

```
.\lab.ps1 verify
```

3.

4. Copy

Cách 3 (Cấu hình vĩnh viễn Code Page UTF-8 trong PowerShell):

```
chcp 65001
```

```
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8
```

```
$env:PYTHONUTF8 = "1"
```

```
$env:PYTHONIOENCODING = "utf-8"
```

```
.\lab.ps1 verify
```

5.

6. Copy

2. Hướng dẫn Demo trên lớp (In-Class Live Demo — Không bắt buộc)

[!NOTE] **Mục đích phần Demo:** Phần demo trực tiếp trên lớp là **tùy chọn (không bắt buộc)**, được thiết kế nhằm giúp học viên rèn luyện tư duy trình bày kỹ thuật (*Engineering Presentation*), xây chuỗi bức tranh toàn cảnh từ phần cứng vi mô đến vận hành vĩ mô, và nhận diện sâu sắc ý nghĩa của những con số đo đạc được trên chính chiếc máy tính của mình gắn liền với lý thuyết Day 20.

1. Giới thiệu Hạ tầng

(make probe & hardware.json)

2. Trình diễn Baseline Latency

(TTFT vs TPOT & Trade-off 2-bit/4-bit)

3. Giải thích Tối ưu Luồng

(Thread Sweep & Cơ chế P/E Cores)

4. Live Load Test & Continuous Batching

(make serve + make load-50 + make metrics)

5. Phân tích Bảo hòa & RAG Pipeline

(Little's Law, Queue Time & Prefill $O(N^2)$)

Nội dung trình bày từng bước (Thời lượng gợi ý: 3 – 5 phút):

1. Giới thiệu Hạ tầng máy của bạn (make probe / .\lab.ps1 probe):

- Mở Terminal chạy `make probe` $\rightarrow$ Nêu loại CPU, số nhân, RAM và backend gia tốc (Metal/CUDA/Vulkan).
- *Ý nghĩa:* Khẳng định tính tương thích và lý do lựa chọn mô hình Gemma 4 E2B hay Qwen3.5.

2. Trình diễn Baseline Latency (`make bench / .\lab.ps1 bench`):

- Mở bảng kết quả `benchmarks/01-quickstart-results.md`.
- *Nội dung nói:* Phân biệt rõ **TTFT (Prefill)** và **TPOT (Decode)**. Chỉ ra bản 2-bit decode nhanh hơn bao nhiêu % và giải thích vì sao trong production ta vẫn ưu tiên 4-bit để giữ vững chất lượng suy luận.

3. Giải thích cú nhảy hiệu năng quan trọng nhất (`make tune / .\lab.ps1 tune`):

- Mở bảng quét luồng `benchmarks/01-tuning-tg128.md`.
- *Nội dung nói:* Chỉ ra điểm ngọt ngào (Sweet spot `-t`). Giải thích **cơ chế kiến trúc CPU** (hiện tượng thất cổ chai do đồng bộ nhân E hoặc tranh chấp L2 cache) đã mang lại speedup thực tế thế nào.

4. Live Demo Continuous Batching dưới tải cao:

- Bật `make serve` ở Terminal 1 $\rightarrow$ Bật `make load-50` ở Terminal 2 $\rightarrow$ Bật `make metrics` ở Terminal 3.
- *Nội dung nói:* Chỉ vào giá trị `Peak n_busy_slots_per_decode` tiệm cận $\$4/4\$$ slots trên Terminal 3 $\rightarrow$ Chứng minh cho cả lớp thấy **Continuous Batching** đang thực sự nhồi nhiều request vào cùng 1 bước decode GPU.

5. Phân tích Bảo hòa Hàng đợi & Bài học RAG (`make load-report & make sweep-ctx`):

- Mở `benchmarks/02-server-results.md` $\rightarrow$ Áp dụng **Định luật Little** để giải thích vì sao khi tải tăng 5x thì độ trễ P95 phình to chủ yếu là do **Queue Time**, dẫn đến **Goodput@SLO** sụt giảm.
- Mở `benchmarks/bonus-ctx-len-sweep.md` $\rightarrow$ Chỉ ra chi phí **Prefill $\$O(N^2)\$$** tăng vọt lên hàng chục giây khi context dài, khẳng định tầm quan trọng của *Chunked Prefill* và *Prompt Caching* trong các hệ thống RAG thực tế.

6. Nộp bài

Danh mục Artefacts bắt buộc trong Git:

`hardware.json` & `models/active.json`.

Thư mục `benchmarks/` chứa đầy đủ các file kết quả và phân tích của bạn.

Thư mục `submission/screenshots/` chứa đủ 5 ảnh chụp màn hình định dạng `.png`.

Báo cáo chi tiết [submission/REFLECTION.md](#) đã điền đủ các mục §1 đến §8.

Không commit file model nặng `models/*.gguf` và thư mục `runtime/`.

Quy trình nộp trên CodeLab:

Commit toàn bộ thay đổi lên Git:

```
git add hardware.json models/active.json benchmarks/ submission/
```

```
git commit -m "Complete Day 20 Lab Submission: Base + Bonus B2"
```

```
git push origin main
```

- 1.
2. Copy
3. **Chuyển chế độ Repository trên GitHub sang PUBLIC.**
4. Nộp đường dẫn (URL) Repository GitHub công khai **trên CodeLab.**

Góp ý cho buổi Lab

Không bắt buộc và không ảnh hưởng việc nộp bài. Giảng viên chỉ xem phản hồi ẩn danh.

Góp ý bài Lab

Nộp bài và đánh giá Lab

Dán link GitHub, Drive hoặc LMS của bài đã nộp. Điểm và nhận xét sẽ không hiển thị tại đây.

Đang tải trạng thái bài nộp...

VLearn Codelabs | Học AI thực chiến qua Lab